# Student Store-front

## Engineering a Campus-Specific Bridge

**The Goal:** Moving from chaotic WhatsApp groups to a clean, verified marketplace.
**Implementation Focus:** How we pivoted from a "scraper" to a "bot bridge" using Spring Boot & Node.js.

# 1. Implementation: The Voluntary Forwarding Bridge

> "What magic automation does your framework provide?"

- **The Pivot:** Feedback warned us that scrapers are fragile and legally risky.
- **The Solution:** A **Voluntary Bot Bridge** ( `whatsapp-web.js` ).
- **Implementation Detail:**
  - Instead of only scraping, users can **forward** a message to our bot.
  - The Node.js service listens for the `message_create` event.
  - It extracts structured data (using regex/Gemini) and POSTs it to: `POST /api/posts` with a `sellerId` mapped from the sender's phone number.

# 2. Spring Boot "Clean Board" Backend

> "What's the simplest version that makes people switch?"

- **Implementation Choice:** Kotlin + Spring Data JPA.
- **The Goal:** Real-time status updates ("Sold" vs "Available").
- **Code Snippet (PostRepository.kt):**

```kotlin
interface PostRepository: JpaRepository<Post, Long> {
    // Direct database-level filtering for the storefront
    fun findByIsSoldFalse(pageable: Pageable): Page<Post>
}
```

- **Why this beats WhatsApp:** A simple `PATCH /api/posts/{id}/mark-sold` instantly removes the item from the feed. No more "Is this still available?" comments.

# 3. Local Trust: Campus-Specific Security

> "What's the local angle Rumie can't handle?"

- **Feature:** `@constructor.university` email verification.
- **The Implementation:**
  - **Security Layer:** Spring Security + JWT.
  - **Trust Indicator:** A "Verified" flag in the DB, only set if the email matches the campus domain.
  - **Local UI:** The PWA shows a distinct badge. Rumie is too generic; we integrate directly with our specific university's email system for 1-tap trust.

# Creative Decision: "WhatsApp-as-UI"

> "What creative architectural decision did you make?"

- **Decision:** The backend is the source of truth, but WhatsApp is the **Admin Panel**.
- **Implementation:**
  - When a post is created, the bot replies with a **Magic Link**.
  - **Magic Link Logic:** A short-lived JWT sent as a URL parameter.
  - `GET /storefront?token=eyJhbGci...`
  - This logs the user into the PWA dashboard *without* a password, making status updates as easy as opening a link from their chat history.

# Technical Disappointment: JPA Pagination vs. Real-time

> "How did you get disappointed with some tool?"

- **The Tool:** Standard JPA `findAll()`.
- **The Issue:** With hundreds of campus posts, raw lists are slow.
- **The Fix:** We had to implement `Pageable` in every controller.

```
@GetMapping("/available")
fun getAvailablePosts(@PageableDefault(size = 20) pageable: Pageable): ResponseEntity<Page<PostResponseDTO>>
```

- **Learning:** Even for a "simple" board, performance requires thinking about data transfer early on.

# Architecture Summary

| Layer | Technology | Key Implementation |
|---|---|---|
| **Edge** | Node.js | Voluntary Forwarding Bot |
| **Core** | Kotlin / Spring Boot | Post-Category State Management |
| **Auth** | JWT / Magic Links | University Email Domain Check |
| **Store** | PostgreSQL | `isSold` state synchronization |

# Lessons Learned

1. **Pivoting is key:** Feedback on scrapers saved us from a legal/technical dead-end.
2. **Minimalism works:** A clean, paginated board is more useful than a "super app" that nobody knows how to use.
3. **Local wins:** Authenticating against `@constructor.university` is our "killer feature."

# Thank You!

## Questions?